

MySQL

Complete Study Guide

Beginner-Friendly · Interview-Ready · Revision-Optimised

Topics Covered

DDL · DML · DQL · WHERE · ORDER BY · GROUP BY · Aggregate Functions
JOINS · Subqueries · Window Functions · Constraints · Views
Indexes · TCL Commands · Stored Procedures

Generated: 6 May 2026

Shweta Jadhav

1. Introduction to SQL & MySQL

1.1 What is SQL?

SQL (Structured Query Language) is a standard language used to communicate with relational databases. It allows users to create, retrieve, manipulate, and control data stored in tables.

Feature	Description
Full Form	Structured Query Language
Pronounced	"See-quel" or "S-Q-L"
Standard Body	ISO / ANSI
First Appeared	1974 (IBM)
Used With	MySQL, PostgreSQL, SQL Server, Oracle, SQLite

1.2 What is MySQL?

MySQL is an open-source **Relational Database Management System (RDBMS)** that uses SQL as its query language. It stores data in tables with rows and columns, and relationships are formed between tables using keys.

Property	SQL	MySQL
Type	Language	Software (RDBMS)
Open Source	N/A	Yes (Community Edition)
Created By	IBM (1974)	MySQL AB (1995)
Now Owned By	ISO Standard	Oracle Corporation
Use Case	Query databases	Manage & store data

1.3 SQL Command Categories

SQL commands are grouped into five categories:

Category	Full Form	Commands	Purpose
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE	Define database structure
DML	Data Manipulation Language	INSERT, UPDATE, DELETE	Manipulate data in tables
DQL	Data Query Language	SELECT	Retrieve/query data
DCL	Data Control Language	GRANT, REVOKE	Control access permissions

Category	Full Form	Commands	Purpose
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT	Manage transactions

1.4 Key Points

- **Case-insensitive:** SQL keywords are case-insensitive (SELECT = select), but best practice is UPPERCASE for keywords.
- Statements end with a **semicolon (;)** in MySQL.
- MySQL stores data in tables inside a database (schema).
- Every table has rows (records) and columns (fields/attributes).
- **Primary Key** uniquely identifies each row; **Foreign Key** links two tables.

2. DDL — Data Definition Language

Definition: DDL commands define or modify the structure (schema) of database objects such as databases, tables, and indexes. DDL changes are **auto-committed** — they cannot be rolled back.

2.1 CREATE

Definition

Used to create a new database, table, index, or view.

Syntax & Example

```
-- Create a database
CREATE DATABASE company_db;
USE company_db;

-- Create a table
CREATE TABLE employees (
    emp_id    INT          PRIMARY KEY AUTO_INCREMENT,
    name      VARCHAR(100) NOT NULL,
    dept      VARCHAR(50),
    salary    DECIMAL(10,2) DEFAULT 0.00,
    joined_on DATE
);
```

- **AUTO_INCREMENT** — MySQL automatically generates a unique number for each new row.
- **VARCHAR(n)** — variable-length string up to n characters.
- **DECIMAL(10,2)** — numeric value with 10 total digits and 2 decimal places.

2.2 ALTER

Definition

Used to modify an existing table structure — add, drop, or modify columns; add or drop constraints.

Syntax & Examples

```
-- Add a new column
ALTER TABLE employees ADD COLUMN email VARCHAR(150);

-- Modify column data type
ALTER TABLE employees MODIFY COLUMN salary DECIMAL(12,2);

-- Rename a column (MySQL 8+)
ALTER TABLE employees RENAME COLUMN dept TO department;

-- Drop a column
ALTER TABLE employees DROP COLUMN email;
```

2.3 DROP

Definition

Permanently deletes a database object (table, database, index, view). All data and structure are lost. Cannot be undone.

Syntax & Examples

```
DROP TABLE employees;           -- deletes table + all data
DROP DATABASE company_db;        -- deletes entire database
DROP TABLE IF EXISTS temp_tbl;  -- safe: no error if table missing
```

2.4 TRUNCATE

Definition

Removes ALL rows from a table instantly but keeps the table structure intact. Faster than DELETE because it does not log individual row deletions.

Syntax

```
TRUNCATE TABLE employees;
```

Command	Removes Data	Removes Structure	Rollback Possible	Speed
DROP	Yes	Yes	No	Fast
TRUNCATE	Yes	No	No	Very Fast
DELETE	Yes	No	Yes	Slow (logged)

2.5 Key Points & Common Mistakes

- **DDL is auto-committed** — always review before executing DROP or TRUNCATE.
- Use IF EXISTS / IF NOT EXISTS to avoid errors in scripts.
- **TRUNCATE resets AUTO_INCREMENT** counter to 1; DELETE does not.
- **Common Mistake:** Using DROP when TRUNCATE was intended — DROP removes the table itself!

3. DML — Data Manipulation Language

Definition: DML commands add, modify, or remove data inside tables. Unlike DDL, DML statements are **not auto-committed** and can be rolled back within a transaction.

3.1 INSERT

Definition

Adds one or more new rows to a table.

Syntax & Examples

```
-- Insert a single row
INSERT INTO employees (name, department, salary, joined_on)
VALUES ('Priya Sharma', 'Engineering', 75000.00, '2022-03-15');

-- Insert multiple rows at once
INSERT INTO employees (name, department, salary, joined_on) VALUES
('Ravi Kumar', 'HR', 55000, '2021-06-01'),
('Anjali Mehta', 'Marketing', 62000, '2023-01-10'),
('Dev Patel', 'Engineering', 80000, '2020-09-20');
```

- Column list is optional if you supply values for ALL columns in order.
- Inserting multiple rows in one statement is faster than multiple single inserts.

3.2 UPDATE

Definition

Modifies existing data in one or more rows.

Syntax & Examples

```
-- Update salary for a specific employee
UPDATE employees
SET salary = 85000
WHERE emp_id = 1;

-- Give 10% raise to all Engineering staff
UPDATE employees
SET salary = salary * 1.10
WHERE department = 'Engineering';
```

⚠ Warning: Always use a WHERE clause with UPDATE. Without it, every row in the table is updated!

3.3 DELETE

Definition

Removes one or more rows from a table. Each deletion is logged, so it can be rolled back.

Syntax & Examples

```
-- Delete a specific employee
DELETE FROM employees WHERE emp_id = 3;

-- Delete all employees in a department
DELETE FROM employees WHERE department = 'Marketing';

-- Delete ALL rows (but keep structure)
DELETE FROM employees;    -- equivalent to TRUNCATE but slower
```

- **Common Mistake:** Forgetting WHERE clause — deletes ALL rows!
- DELETE supports WHERE; TRUNCATE does not.
- DELETE fires BEFORE/AFTER DELETE triggers; TRUNCATE does not.

4. DQL — Data Query Language (SELECT)

Definition: The SELECT statement retrieves data from one or more tables. It is the most frequently used SQL command.

4.1 Basic Syntax

```
SELECT column1, column2, ...  
FROM   table_name  
[WHERE condition]  
[GROUP BY column]  
[HAVING condition]  
[ORDER BY column [ASC|DESC]]  
[LIMIT n];
```

4.2 Examples

```
-- Select all columns  
SELECT * FROM employees;  
  
-- Select specific columns  
SELECT name, department, salary FROM employees;  
  
-- Column alias  
SELECT name AS 'Employee Name', salary AS 'Monthly Pay'  
FROM employees;  
  
-- Remove duplicates  
SELECT DISTINCT department FROM employees;  
  
-- Limit rows returned  
SELECT * FROM employees LIMIT 5;  
SELECT * FROM employees LIMIT 5 OFFSET 10; -- skip first 10
```

4.3 SELECT Execution Order

SQL clauses are written in a specific order but executed in a different order:

Step	Clause	What Happens
1	FROM / JOIN	Identify and join source tables
2	WHERE	Filter rows (before grouping)
3	GROUP BY	Group filtered rows
4	HAVING	Filter groups
5	SELECT	Choose which columns to return
6	DISTINCT	Remove duplicates

Step	Clause	What Happens
7	ORDER BY	Sort results
8	LIMIT / OFFSET	Restrict number of rows

4.4 Key Points

- **SELECT *** fetches all columns — avoid in production for performance reasons.
- **DISTINCT** applies to the entire row, not just one column.
- **AS** creates an alias — the alias can be used in ORDER BY but NOT in WHERE.
- **LIMIT** is MySQL-specific (other RDBMS use TOP or FETCH FIRST).

5. WHERE Clause & Operators

Definition: The WHERE clause filters rows based on a condition. Only rows that satisfy the condition are included in the result.

5.1 Comparison Operators

Operator	Meaning	Example
=	Equal to	WHERE salary = 50000
!= or <>	Not equal to	WHERE dept != 'HR'
>	Greater than	WHERE salary > 60000
<	Less than	WHERE salary < 30000
>=	Greater than or equal	WHERE salary >= 50000
<=	Less than or equal	WHERE salary <= 70000

5.2 Logical Operators

```
-- AND: both conditions must be true
SELECT * FROM employees WHERE dept = 'Engineering' AND salary > 70000;

-- OR: at least one condition must be true
SELECT * FROM employees WHERE dept = 'HR' OR dept = 'Marketing';

-- NOT: negates a condition
SELECT * FROM employees WHERE NOT dept = 'HR';
```

5.3 Special Operators

Operator	Purpose	Example
BETWEEN ... AND	Range check (inclusive)	WHERE salary BETWEEN 50000 AND 80000
IN (...)	Match any value in list	WHERE dept IN ('HR','Finance','IT')
NOT IN (...)	Exclude values in list	WHERE dept NOT IN ('HR')
LIKE	Pattern matching	WHERE name LIKE 'A%'
NOT LIKE	Exclude pattern matches	WHERE name NOT LIKE '%Kumar'
IS NULL	Check for NULL	WHERE email IS NULL
IS NOT NULL	Check for non-NULL	WHERE email IS NOT NULL

5.4 LIKE Wildcards

Wildcard	Meaning & Example
%	Any sequence of characters — LIKE 'A%' matches Amit, Anjali, Arjun
_	Any single character — LIKE '_avi' matches Ravi, Davi
ESCAPE	Escape a wildcard — LIKE '50\%' ESCAPE '\' matches the literal text '50%'

5.5 Key Points

- NULL cannot be compared with = or !=; always use IS NULL / IS NOT NULL.
- **BETWEEN** is inclusive: BETWEEN 10 AND 20 includes both 10 and 20.
- Operator precedence: NOT > AND > OR — use parentheses to be explicit.
- **Common Mistake:** Writing WHERE name = NULL instead of WHERE name IS NULL.

6. ORDER BY

Definition: Sorts the result set by one or more columns in ascending (ASC) or descending (DESC) order. Default is ASC.

Why use it: To present data in a meaningful, readable order — e.g., top earners, newest records, alphabetical lists.

6.1 Syntax & Examples

```
-- Sort by salary descending (highest first)
SELECT name, salary FROM employees ORDER BY salary DESC;

-- Sort by department alphabetically, then by salary descending
SELECT name, department, salary
FROM   employees
ORDER BY department ASC, salary DESC;

-- Sort by column position (1-based index)
SELECT name, salary FROM employees ORDER BY 2 DESC;  -- 2 = salary

-- NULLs: in MySQL, NULLs sort FIRST in ASC, LAST in DESC
SELECT * FROM employees ORDER BY email ASC;
```

6.2 Key Points

- ORDER BY is always the last clause to execute (before LIMIT).
- You can ORDER BY a column that is not in the SELECT list.
- Sorting by multiple columns: leftmost column is the primary sort key.
- **Common Mistake:** Expecting a specific order without ORDER BY — SQL tables have no guaranteed default order.

7. GROUP BY & HAVING

7.1 GROUP BY

Definition: Groups rows that have the same value in specified columns into summary rows. Always used with aggregate functions.

Real-world analogy: "Give me the total sales per city" -- GROUP BY city.

Syntax & Example

```
-- Count employees per department
SELECT  department, COUNT(*) AS headcount, AVG(salary) AS avg_salary
FROM    employees
GROUP BY department;

-- Output:
-- department | headcount | avg_salary
-- Engineering | 2         | 77500.00
-- HR          | 1         | 55000.00
-- Marketing   | 1         | 62000.00
```

7.2 HAVING

Definition: Filters groups after GROUP BY — similar to WHERE but applied to aggregated results.

Clause	Filters
WHERE	Individual rows (before grouping)
HAVING	Groups / aggregated results (after grouping)

Example

```
-- Departments with more than 1 employee AND avg salary above 60,000
SELECT  department, COUNT(*) AS headcount, AVG(salary) AS avg_sal
FROM    employees
GROUP BY department
HAVING  COUNT(*) > 1 AND AVG(salary) > 60000;
```

7.3 Key Points

- Every non-aggregated column in SELECT must appear in GROUP BY.
- HAVING can reference aggregate functions; WHERE cannot.
- **Common Mistake:** Using HAVING instead of WHERE to filter individual rows — this forces MySQL to aggregate all rows first.
- You can use both WHERE and HAVING in the same query: WHERE filters rows first, then GROUP BY groups them, then HAVING filters the groups.

8. Aggregate Functions

Definition: Aggregate functions perform a calculation on a set of rows and return a single value. They are typically used with GROUP BY.

Function	Purpose	Syntax Example	Result
COUNT(*)	Count all rows	SELECT COUNT(*) FROM employees;	4
COUNT(col)	Count non-NULL values	SELECT COUNT(email) FROM employees;	Varies
SUM(col)	Total of numeric column	SELECT SUM(salary) FROM employees;	272000
AVG(col)	Average of numeric column	SELECT AVG(salary) FROM employees;	68000
MAX(col)	Largest value	SELECT MAX(salary) FROM employees;	80000
MIN(col)	Smallest value	SELECT MIN(salary) FROM employees;	55000

8.1 Practical Examples

```
-- Salary statistics per department
SELECT
    department,
    COUNT(*)      AS total_emp,
    SUM(salary)   AS total_salary,
    AVG(salary)   AS avg_salary,
    MAX(salary)   AS highest_salary,
    MIN(salary)   AS lowest_salary
FROM employees
GROUP BY department
ORDER BY total_emp DESC;
```

8.2 Key Points

- **COUNT(*)** counts all rows including NULLs; **COUNT(col)** ignores NULLs.
- SUM, AVG, MAX, MIN all ignore NULL values automatically.
- **AVG** returns a decimal; consider ROUND(AVG(salary), 2).
- Aggregate functions cannot be used in WHERE — use HAVING instead.
- **Common Mistake:** SELECT name, COUNT(*) FROM employees — name is not in GROUP BY, which is invalid in strict SQL mode.

9. JOINS

Definition: A JOIN combines rows from two or more tables based on a related column (usually a foreign key relationship).

9.1 Sample Tables

```
-- employees table
emp_id | name           | dept_id
-----+-----+-----
1      | Priya Sharma   | 10
2      | Ravi Kumar     | 20
3      | Anjali Mehta   | 30
4      | Dev Patel      | NULL

-- departments table
dept_id | dept_name
-----+-----
10      | Engineering
20      | HR
30      | Marketing
40      | Finance
```

9.2 JOIN Types

INNER JOIN — Returns only matching rows from both tables

Venn Diagram: Only the INTERSECTION — rows that exist in BOTH tables

```
SELECT e.name, d.dept_name
FROM   employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;

-- Result: Priya (Engineering), Ravi (HR), Anjali (Marketing)
-- Dev Patel excluded (dept_id is NULL — no match)
-- Finance excluded (no employee linked to it)
```

LEFT JOIN (LEFT OUTER JOIN) — All rows from left table + matching right

Venn Diagram: Everything in LEFT table; NULLs fill in where right table has no match

```
SELECT e.name, d.dept_name
FROM   employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;

-- Result: Priya (Engineering), Ravi (HR), Anjali (Marketing), Dev (NULL)
-- Dev Patel included with dept_name = NULL
```

RIGHT JOIN (RIGHT OUTER JOIN) — All rows from right table + matching left

Venn Diagram: Everything in RIGHT table; NULLs fill in where left table has no match

```
SELECT e.name, d.dept_name
FROM   employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;
```

```
-- Result: Priya, Ravi, Anjali + Finance (name = NULL)
-- Finance department included with name = NULL
```

FULL OUTER JOIN — All rows from both tables (MySQL uses UNION)

Venn Diagram: Everything from BOTH tables; NULLs where there is no match on either side

```
-- MySQL does not support FULL OUTER JOIN directly; use UNION:
SELECT e.name, d.dept_name
FROM   employees e LEFT JOIN departments d ON e.dept_id = d.dept_id
UNION
SELECT e.name, d.dept_name
FROM   employees e RIGHT JOIN departments d ON e.dept_id = d.dept_id;
```

9.3 JOIN Comparison Table

JOIN Type	Left Unmatched	Right Unmatched	Use When
INNER JOIN	Excluded	Excluded	Only want matched data
LEFT JOIN	Included (NULL)	Excluded	All left + optional right data
RIGHT JOIN	Excluded	Included (NULL)	All right + optional left data
FULL OUTER	Included (NULL)	Included (NULL)	Complete picture of both tables

9.4 Self Join

A table joined with itself — useful for hierarchical data like manager–employee relationships.

```
-- employees table has a manager_id column pointing to emp_id
SELECT e.name AS employee, m.name AS manager
FROM   employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

9.5 Key Points

- Always use table aliases (e, d) for readability in multi-table queries.
- **ON vs USING:** ON e.dept_id = d.dept_id is equivalent to USING(dept_id) when column names match.
- JOIN without ON creates a Cartesian product — every row combined with every other row.
- **Common Mistake:** Forgetting the ON clause — causes a Cartesian product (potentially millions of rows).

10. Subqueries

Definition: A subquery is a query nested inside another SQL statement. The inner query executes first; its result is used by the outer query.

10.1 Single-Row Subquery

Returns exactly one row and one column. Used with =, >, <, etc.

```
-- Find employees earning more than the company average salary
SELECT name, salary
FROM   employees
WHERE  salary > (SELECT AVG(salary) FROM employees);
```

10.2 Multi-Row Subquery

Returns multiple rows. Used with IN, ANY, ALL, EXISTS.

```
-- Find employees who work in departments based in Mumbai
SELECT name FROM employees
WHERE dept_id IN (
    SELECT dept_id FROM departments WHERE city = 'Mumbai'
);

-- ANY: salary greater than ANY junior employee
SELECT name FROM employees
WHERE salary > ANY (SELECT salary FROM juniors);

-- ALL: salary greater than ALL junior employees
SELECT name FROM employees
WHERE salary > ALL (SELECT salary FROM juniors);
```

10.3 Correlated Subquery

References a column from the outer query — executes once for each outer row (slower but powerful).

```
-- Find employees earning more than the average in their own department
SELECT e1.name, e1.salary, e1.department
FROM   employees e1
WHERE  e1.salary > (
    SELECT AVG(e2.salary)
    FROM   employees e2
    WHERE  e2.department = e1.department -- references outer query
);
```

10.4 EXISTS / NOT EXISTS

```
-- Find departments that have at least one employee
SELECT dept_name
FROM   departments d
WHERE  EXISTS (
    SELECT 1 FROM employees e WHERE e.dept_id = d.dept_id
);
```

```
) ;
```

10.5 Subquery Locations

Location	Example Use
In WHERE	WHERE salary > (SELECT MAX...)
In FROM	FROM (SELECT ...) AS alias
In SELECT	SELECT name, (SELECT MAX(sal) FROM ...) AS max_sal FROM ...
In HAVING	HAVING AVG(salary) > (SELECT AVG(salary) FROM ...)

10.6 Key Points

- **Performance tip:** Prefer JOINS over correlated subqueries when possible — JOINS are generally faster.
- EXISTS is more efficient than IN when the subquery result is large.
- **Common Mistake:** Using = with a multi-row subquery — causes an error. Use IN instead.

11. Window Functions

Definition: Window functions perform calculations across a set of table rows related to the current row — similar to aggregate functions but **without collapsing rows** into a single result. They use the `OVER()` clause.

Real-world use: Ranking employees, running totals, comparing each row to its group average.

11.1 Syntax

```
function_name() OVER (
    [PARTITION BY column]    -- divide rows into groups (windows)
    [ORDER BY column]        -- order within each partition
    [ROWS/RANGE frame]       -- define the sliding window frame
)
```

11.2 ROW_NUMBER

Assigns a unique sequential number to each row within a partition (no ties).

```
SELECT name, department, salary,
       ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC)
       AS row_num
FROM employees;

-- Result (Engineering dept rows only):
-- name      | salary | row_num
-- Dev Patel  | 80000  | 1
-- Priya Sharma | 75000  | 2
```

11.3 RANK

Assigns a rank to rows; ties receive the same rank, but the next rank is skipped.

```
SELECT name, salary,
       RANK() OVER (ORDER BY salary DESC) AS salary_rank
FROM employees;

-- If two employees have salary 80000:
-- Rank: 1, 1, 3 (rank 2 is skipped)
```

11.4 DENSE_RANK

Same as `RANK` but no gaps — consecutive ranks even for ties.

```
SELECT name, salary,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rank
FROM employees;

-- If two employees have salary 80000:
-- Dense Rank: 1, 1, 2 (no gap)
```

11.5 Ranking Functions Comparison

Salary	ROW_NUMBER	RANK	DENSE_RANK
90000	1	1	1
80000	2	2	2
80000	3	2	2
70000	4	4	3
60000	5	5	4

11.6 Other Useful Window Functions

Function	Description & Use
LAG(col, n)	Value from n rows before the current row — compare to previous period
LEAD(col, n)	Value from n rows after — compare to next period
NTILE(n)	Divide rows into n equal buckets (quartiles, deciles)
SUM() OVER(...)	Running total without collapsing rows
AVG() OVER(...)	Moving average

```
-- Running total of salary
SELECT name, salary,
       SUM(salary) OVER (ORDER BY emp_id) AS running_total
FROM employees;
```

11.7 Key Points

- **PARTITION BY** is optional — without it, the entire result set is one window.
- Window functions are processed after WHERE, GROUP BY, and HAVING.
- **Common Mistake:** Trying to use a window function in a WHERE clause — wrap the query in a subquery / CTE first.

12. Constraints

Definition: Constraints are rules enforced on columns to maintain data integrity and consistency in a database.

Constraint	Purpose	Column / Table
PRIMARY KEY	Uniquely identifies each row	Column/Table
FOREIGN KEY	Links to another table's PK	Table
UNIQUE	Ensures all values are distinct	Column
NOT NULL	Prevents NULL values	Column
CHECK	Validates values against a rule	Column/Table
DEFAULT	Sets a default value	Column

12.1 PRIMARY KEY

Uniquely identifies each row. Cannot be NULL. Only one per table (can be composite).

```
CREATE TABLE employees (
    emp_id INT PRIMARY KEY AUTO_INCREMENT, -- inline
    name   VARCHAR(100) NOT NULL
);

-- Composite primary key (table-level):
CREATE TABLE order_items (
    order_id   INT,
    product_id INT,
    PRIMARY KEY (order_id, product_id)
);
```

12.2 FOREIGN KEY

References the PRIMARY KEY of another table, establishing a parent-child relationship. Prevents orphan records.

```
CREATE TABLE orders (
    order_id   INT PRIMARY KEY AUTO_INCREMENT,
    emp_id     INT,
    order_date DATE,
    FOREIGN KEY (emp_id) REFERENCES employees(emp_id)
        ON DELETE CASCADE -- delete orders if employee deleted
        ON UPDATE CASCADE -- update if parent key changes
);
```

Referential Action	Behaviour
CASCADE	Propagates the change to child rows
SET NULL	Sets FK to NULL in child rows

Referential Action	Behaviour
RESTRICT	Prevents parent delete/update if child rows exist
NO ACTION	Like RESTRICT (default in MySQL)

12.3 UNIQUE

Ensures all values in a column are distinct. Unlike PRIMARY KEY, allows one NULL.

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    email VARCHAR(150) UNIQUE,  
    pan CHAR(10) UNIQUE  
);
```

12.4 NOT NULL

Ensures a column cannot store a NULL value. Applied at column level.

```
name VARCHAR(100) NOT NULL,  
salary DECIMAL(10,2) NOT NULL DEFAULT 0.00
```

12.5 CHECK

Validates data against a user-defined condition (MySQL 8.0.16+ enforces CHECK constraints).

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    salary DECIMAL(10,2) CHECK (salary >= 0),  
    age INT CHECK (age BETWEEN 18 AND 65)  
);
```

12.6 DEFAULT

Provides a default value when no value is supplied during INSERT.

```
status VARCHAR(20) DEFAULT 'Active',  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

12.7 Key Points

- A table can have only ONE PRIMARY KEY but multiple UNIQUE constraints.
- FOREIGN KEY columns should generally be indexed for performance.
- **Common Mistake:** Inserting a FOREIGN KEY value that doesn't exist in the parent table — causes a constraint violation error.

13. Views

Definition: A view is a **virtual table** based on a SELECT query. It does not store data physically — it stores the query definition. Every time you query a view, MySQL executes the underlying SELECT.

Why use Views: Simplify complex queries, restrict column-level access, provide a stable interface when table structure changes.

13.1 Creating a View

```
-- Create a view showing employee details with dept name
CREATE VIEW employee_summary AS
SELECT e.emp_id, e.name, d.dept_name, e.salary
FROM   employees e
JOIN   departments d ON e.dept_id = d.dept_id;

-- Use the view just like a table
SELECT * FROM employee_summary WHERE salary > 70000;
```

13.2 Modifying & Dropping Views

```
-- Replace an existing view
CREATE OR REPLACE VIEW employee_summary AS
SELECT e.emp_id, e.name, d.dept_name, e.salary, e.joined_on
FROM employees e JOIN departments d ON e.dept_id = d.dept_id;

-- Drop a view
DROP VIEW IF EXISTS employee_summary;
```

13.3 Updatable Views

A view is updatable (can be used in INSERT/UPDATE/DELETE) only if it is based on a single table without DISTINCT, GROUP BY, HAVING, UNION, or aggregate functions.

13.4 Key Points

- **Security:** Grant users access to views instead of base tables to restrict sensitive columns.
- Views always reflect current data — they are not snapshots.
- Complex views (with JOINS, aggregates) are generally not updatable.
- **Common Mistake:** Assuming views improve query performance — they don't (unless materialized views are used, which MySQL does not natively support).

14. Indexes

Definition: An index is a database structure that speeds up data retrieval operations. It works like a book's index — instead of scanning every page, MySQL looks up the index to find the data location. Trade-off: Indexes speed up SELECT but slow down INSERT, UPDATE, DELETE (index must also be updated).

14.1 Creating Indexes

```
-- Create an index on a single column
CREATE INDEX idx_department ON employees(department);

-- Create a composite index (multi-column)
CREATE INDEX idx_dept_salary ON employees(department, salary);

-- Create a unique index
CREATE UNIQUE INDEX idx_email ON employees(email);

-- View indexes on a table
SHOW INDEX FROM employees;

-- Drop an index
DROP INDEX idx_department ON employees;
```

14.2 Index Types

Index Type	Description
PRIMARY	Automatically created for PRIMARY KEY — clustered index in InnoDB
UNIQUE	Automatically created for UNIQUE constraint — prevents duplicates
INDEX / KEY	General-purpose non-unique index for faster lookups
FULLTEXT	For text search (MATCH ... AGAINST) on large text columns
SPATIAL	For geometric/geographic data

14.3 When to Use Indexes

- Columns frequently used in WHERE, JOIN ON, or ORDER BY clauses.
- Foreign key columns — MySQL does NOT auto-create FK indexes.
- High-cardinality columns (many distinct values like email, phone).
- **Avoid:** Indexing small tables, columns with very few distinct values (e.g., gender), or columns rarely used in queries.
- **Common Mistake:** Over-indexing a write-heavy table — every index slows down INSERT/UPDATE/DELETE.

15. TCL — Transaction Control Language

Definition: TCL commands manage **transactions** — a sequence of SQL operations executed as a single logical unit. Either ALL operations succeed (commit) or ALL are undone (rollback).

Real-world example: Bank transfer — debit account A AND credit account B must both succeed or neither happens.

15.1 ACID Properties

Property	Meaning	Example
Atomicity	All or nothing	Transfer completes fully or not at all
Consistency	Data stays valid	Balance never goes negative
Isolation	Transactions don't interfere	Concurrent transfers don't corrupt data
Durability	Committed data persists	Power failure won't lose committed data

15.2 COMMIT

Permanently saves all changes made in the current transaction.

```
START TRANSACTION;
  UPDATE accounts SET balance = balance - 5000 WHERE acc_id = 101;
  UPDATE accounts SET balance = balance + 5000 WHERE acc_id = 202;
COMMIT; -- both updates are saved permanently
```

15.3 ROLLBACK

Undoes all changes made in the current transaction — returns data to its previous state.

```
START TRANSACTION;
  UPDATE accounts SET balance = balance - 5000 WHERE acc_id = 101;
  -- Something went wrong!
ROLLBACK; -- debit is reversed; balance restored
```

15.4 SAVEPOINT

Creates a checkpoint within a transaction so you can roll back to a specific point without undoing everything.

```
START TRANSACTION;
  INSERT INTO orders (customer, total) VALUES ('Priya', 2500);
  SAVEPOINT sp_after_order;           -- checkpoint
  INSERT INTO order_items VALUES (1, 'Laptop', 1);
  -- Error in items! Roll back only the items insert
  ROLLBACK TO SAVEPOINT sp_after_order;
  -- Order header remains; only items insert is undone
COMMIT;
```

15.5 Auto-commit Mode

```
-- Check auto-commit mode (ON by default in MySQL)
SELECT @@autocommit;    -- 1 = ON

-- Disable auto-commit
SET autocommit = 0;
```

15.6 Key Points

- **DDL statements (CREATE, DROP, ALTER, TRUNCATE)** cause an implicit COMMIT — they cannot be rolled back.
- DML statements (INSERT, UPDATE, DELETE) can be rolled back within a transaction.
- RELEASE SAVEPOINT removes a savepoint but does not commit.
- **Common Mistake:** Expecting to roll back a TRUNCATE — not possible because TRUNCATE is DDL (auto-commit).

16. Stored Procedures (Overview)

Definition: A stored procedure is a named, precompiled block of SQL code stored in the database and executed on demand. It can accept parameters, contain logic (IF, LOOP), and return results.

Why use them: Reusability, reduced network traffic, encapsulation of business logic, improved performance (precompiled).

16.1 Creating a Stored Procedure

```
-- Change delimiter so MySQL doesn't end the procedure early
DELIMITER //

CREATE PROCEDURE GetEmployeesByDept(IN dept_name VARCHAR(50))
BEGIN
    SELECT emp_id, name, salary
    FROM   employees
    WHERE  department = dept_name;
END //

DELIMITER ;

-- Call the procedure
CALL GetEmployeesByDept('Engineering');
```

16.2 Parameters

Parameter Type	Description & Use
IN	Input parameter — value passed in by the caller (default)
OUT	Output parameter — procedure sets a value, caller reads it
INOUT	Both input and output — caller passes a value, procedure modifies it

```
-- Procedure with OUT parameter
DELIMITER //
CREATE PROCEDURE GetDeptSalaryBudget(
    IN  dept_name VARCHAR(50),
    OUT total_budget DECIMAL(15,2)
)
BEGIN
    SELECT SUM(salary) INTO total_budget
    FROM   employees
    WHERE  department = dept_name;
END //
DELIMITER ;

CALL GetDeptSalaryBudget('Engineering', @budget);
SELECT @budget;    -- 155000.00
```

16.3 Managing Stored Procedures

```
SHOW PROCEDURE STATUS WHERE Db = 'company_db'; -- list all procedures
SHOW CREATE PROCEDURE GetEmployeesByDept;      -- view definition
DROP PROCEDURE IF EXISTS GetEmployeesByDept;    -- delete procedure
```

16.4 Key Points

- Stored procedures reduce the amount of SQL code sent across the network.
- They can contain variables (DECLARE), conditional logic (IF/ELSE/CASE), and loops (WHILE/REPEAT/LOOP).
- **Stored Procedures vs Functions:** Functions return a single value and can be used in SELECT; Procedures can return multiple result sets but cannot be used inline in SELECT.
- **Common Mistake:** Forgetting to change the delimiter (DELIMITER //) — causes MySQL to interpret END as the end of the statement prematurely.

Quick Reference — Interview Cheat Sheet

Most Common Interview Questions & Answers

Question	Key Answer
Difference between DELETE, DROP, TRUNCATE?	DELETE: removes rows (DML, rollbackable). TRUNCATE: removes all rows (DDL, fast, resets AI). DROP: removes table+structure (DDL, permanent).
WHERE vs HAVING?	WHERE filters rows before grouping; HAVING filters groups/aggregates after GROUP BY.
INNER vs LEFT JOIN?	INNER: only matched rows. LEFT: all left rows + matched right (NULL where no match).
What is a Primary Key?	Unique, NOT NULL identifier for each row. One per table.
What is a Foreign Key?	Column referencing another table's PK. Maintains referential integrity.
RANK vs DENSE_RANK?	RANK skips numbers after ties (1,1,3). DENSE_RANK does not skip (1,1,2).
What is a View?	A virtual table storing a SELECT query — no physical data storage.
Can we rollback TRUNCATE?	No. TRUNCATE is DDL and auto-commits.
What is a Correlated Subquery?	A subquery that references a column from the outer query; runs once per outer row.
What is normalisation?	Organising tables to reduce redundancy and dependency (1NF, 2NF, 3NF, BCNF).

SQL Keywords Summary

Category	Keywords
DDL	CREATE · ALTER · DROP · TRUNCATE · RENAME
DML	INSERT · UPDATE · DELETE · MERGE
DQL	SELECT · FROM · WHERE · GROUP BY · HAVING · ORDER BY · LIMIT
TCL	START TRANSACTION · COMMIT · ROLLBACK · SAVEPOINT · RELEASE SAVEPOINT
DCL	GRANT · REVOKE
Joins	INNER JOIN · LEFT JOIN · RIGHT JOIN · CROSS JOIN · SELF JOIN
Operators	= · != · > · < · >= · <= · BETWEEN · IN · LIKE · IS NULL · AND · OR · NOT
Aggregates	COUNT · SUM · AVG · MAX · MIN

Category	Keywords
Window	ROW_NUMBER · RANK · DENSE_RANK · NTILE · LAG · LEAD · PARTITION BY
Constraints	PRIMARY KEY · FOREIGN KEY · UNIQUE · NOT NULL · CHECK · DEFAULT

— End of MySQL Study Guide —